

Fundamentos de Projeto e Análise de Algoritmos

— Introdução a Análise de Algoritmos —

Análise de Algoritmo

Analisar um algoritmo significa avaliar os **recursos necessários para sua execução**, como:

- ❖ Memória;
 - ❖ Largura de banda de comunicação;
 - ❖ Hardware;
 - ❖ Tempo de Execução.
-
- ❖ Para um mesmo problema, podem existir vários algoritmos, a análise ajuda a escolher o **mais eficiente**.



Por que estudar análise de algoritmos?



- ❖ Tempo e memória são recursos limitados;
- ❖ Computadores são rápidos, mas não infinitos;
- ❖ Problemas crescem cada vez mais.

Algoritmos eficientes permitem resolver **problemas maiores**.

Por que estudar análise de algoritmos?

- ❖ Suponha que para resolver um determinado problema você tem disponível um algoritmo exponencial (2^n) e um computador capaz de executar 10^4 operações por segundo.

2^n na máquina 10^4		
	tempo(s)	tamanho
	0,1	10
	1	13
1 minuto	60	19
1 hora	3600	25
1 dia	86400	30
1 ano	31536000	38

Por que estudar análise de algoritmos?

- ❖ Compra de um novo computador capaz de executar 10^9 operações por segundo.

		2^n na máquina 10^4	2^n na máquina 10^9
	tempo(s)	tamanho	tamanho
	0,1	10	27
	1	13	30
1 minuto	60	19	36
1 hora	3600	25	42
1 dia	86400	30	46
1 ano	31536000	38	55

Aumento na velocidade computacional tem pouco efeito no tamanho das instâncias resolvidas por algoritmos ineficientes.

Por que estudar análise de algoritmos?

Investir em algoritmo:

- ❖ Você encontrou um algoritmo quadrático (n^2) para resolver o problema.

	tempo(s)	2^n na máquina 10^4	n^2 na máquina 10^4
		tamanho	tamanho
	0,1	10	32
	1	13	100
1 minuto	60	19	775
1 hora	3600	25	6.000
1 dia	86400	30	29.394
1 ano	31536000	38	561.569

Novo algoritmo oferece uma melhoria maior que a compra da nova máquina.

Medida do Tempo de Execução de um Programa

O projeto de algoritmos exige analisar seu **comportamento**.

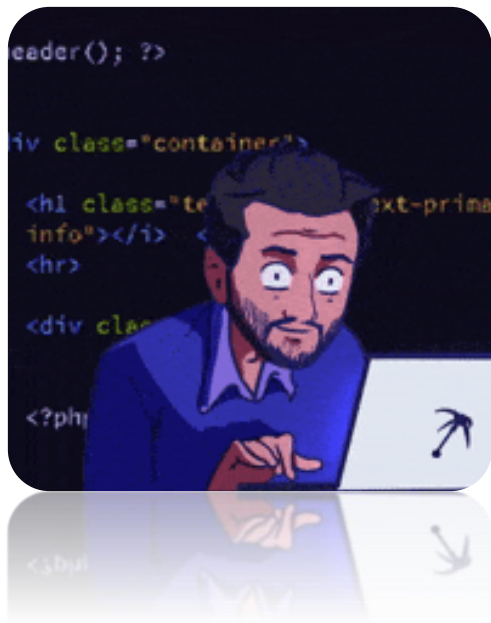
Ao escolher um algoritmo, devemos considerar:

- ❖ Tempo de execução;
- ❖ Uso de memória.



Esses fatores determinam sua **viabilidade prática**.

Medida do Tempo de Execução de um Programa



1. Análise de um algoritmo particular:

Qual é o custo de usar um dado algoritmo para resolver um problema específico?

- ❖ Características que devem ser investigadas:
 - ❖ análise do número de vezes que cada parte do algoritmo deve ser executada;
 - ❖ estudo da quantidade de memória necessária.

Medida do Tempo de Execução de um Programa



2. Análise de uma classe de algoritmos.

Qual é o algoritmo de menor custo possível para resolver um problema particular?

- ❖ Toda uma família de algoritmos é investigada.
- ❖ Procura-se identificar um que seja o melhor possível.

Coloca-se limites para a complexidade computacional dos algoritmos pertencentes à classe.

Custo de um Algoritmo

- ❖ O custo mede o esforço necessário para resolver um problema.
- ❖ O menor custo possível indica a dificuldade do problema.
- ❖ Um algoritmo é **ótimo** quando atinge esse menor custo.



Função de Complexidade

- ❖ Para medir o custo de execução de um algoritmo, define-se uma **função de complexidade**.
- ❖ Seja **$T(n)$** uma função que representa o custo do algoritmo para uma entrada de tamanho **n** .
 - ❖ **n** → tamanho da entrada
 - ❖ **$T(n)$** → quantidade de operações executadas

Na prática, a **complexidade de tempo** não mede segundos, mas o **número de vezes que uma operação relevante é executada**.



Exemplo: Encontrar o Maior Elemento de um Vetor

Considere um vetor com **n** elementos:



Ideia do Algoritmo:

- ❖ Começamos assumindo que o primeiro elemento é o maior.
- ❖ Comparamos com os demais elementos.
- ❖ Atualizamos o maior valor quando necessário.

Quantas comparações são feitas?

$$n - 1$$

Exemplo: Encontrar o Maior Elemento de um Vetor

```
public class Maior {  
    public static int MaiorElemento(int[] v, int n){  
        int maior = v[0];  
        for (int i = 1; i < n; i++)  
            if (maior < v[i])  
                maior = v[i];  
        return maior;  
    }  
}
```

Código: <https://onlinegdb.com/ZQQ6D4KiP>

Exemplo: Encontrar o Maior Elemento de um Vetor

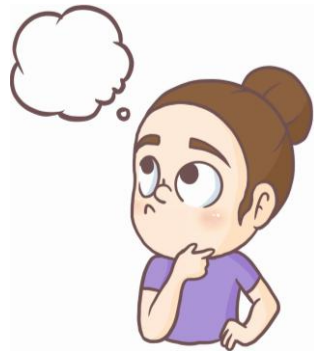
Será que é possível fazer menos comparações?

Para garantir que um elemento é o maior:

- ❖ Ele precisa ser comparado com os outros.
- ❖ Cada elemento (exceto um) deve “perder” pelo menos uma comparação.
- ❖ Se temos n elementos, precisamos de pelo menos: **$n - 1$ comparações**

Conclusão

- ❖ Como o algoritmo faz exatamente $n - 1$ comparações:
 - ❖ Ele usa o número mínimo possível de comparações
 - ❖ Portanto, é um algoritmo ótimo (para essa medida de custo)



A complexidade no desempenho do Algoritmo

O esforço computacional de um algoritmo não pode ser descrito simplesmente por um número, porque a quantidade de trabalho requerido em geral depende da entrada.



Exemplo:

- ❖ a quantidade de trabalho para ordenar uma lista depende do número de elementos da lista;
- ❖ O esforço computacional de um algoritmo costuma depender do **tamanho da entrada**.
- ❖ Mesmo para entradas do mesmo tamanho, o esforço computacional requerido pelo algoritmo pode depender de uma entrada particular.

Critérios de Complexidade

O tempo requerido por um algoritmo sobre uma dada entrada pode ser medido pelo **número de execuções** de algumas operações.

- ❖ Para medir a quantidade de trabalho realizado por um algoritmo, costuma-se escolher uma operação, chamada **operação fundamental** (ou **operação relevante**).
- ❖ A operação escolhida como fundamental deve ser tal que a contagem do número de vezes que ela é executada expresse a quantidade de trabalho do algoritmo, dispensando outras medidas.

Bibliografia



CORMEN, Thomas H et al. **Algoritmos: teoria e prática**. 4. ed. Rio de Janeiro: GEN, c2024.

TOSCANI, Laira Vieira; VELOSO, Paulo A. S. **Complexidade de algoritmos: análise, projeto e métodos**. 3. ed. Porto Alegre: Bookman, 2012. xvii, 262 p.



ZIVIANI, Nivio. **Projeto de algoritmos com implementações em Pascal e C**. 3. ed. rev. e ampl. São Paulo: Cengage Learning, c2011.

